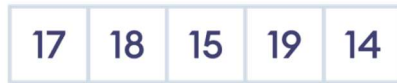


JavaScript Array

An array is an object that can store multiple values at once.

```
const age = [17, 18, 15, 19, 14];
```



Array of age

Why Use Arrays?

Arrays allow us to organize related data by grouping them within a single variable.

Suppose you want to store a list of fruits. Using only variables, this process might look like this:

```
let fruit1 = "Apple";
```

```
let fruit2 = "Banana";
```

```
let fruit3 = "Orange";
```

Here, we've only listed a few fruits. But what if we need to store **100** fruits?

For such a case, the easiest solution is to store them in an array.

```
let fruits = ["Apple", "Banana", "Orange", ...];
```

An array can store many values in a single variable, making it easy to access them by referring to the corresponding index number.

Create an Array

We can create an array by placing elements inside an array literal [], separated by commas.

Example:

```
const numbers = [10, 30, 40, 60, 80];
```

Here,

- numbers - Name of the array.
- [10, 30, 40, 60, 80] - Elements of the array.

Access Elements of an Array

Each element of an array is associated with a number called an **index**, which specifies its position inside the array.

```
let numbers = [10, 30, 40, 60, 80];
```

Here is the indexing of each element:

Indexes →	0	1	2	3	4
	10	30	40	60	80

```
let numbers = [10, 30, 40, 60, 80]
```

```
// access first element
```

```
console.log(numbers[0]); // 10
```

```
// access third element
```

```
console.log(numbers[2]); // 40
```

Change the Elements of an Array

We can add or change elements by accessing the index value. For example,

```
let Activities = [ "eat", "work", "sleep"];
```

```
// change the second element
```

```
// use array index 1
```

```
Activities[1] = "exercise";
```

```
console.log(Activities);
```

```
// Output: [ 'eat', 'exercise', 'sleep' ]
```

Remove Elements From an Array

We can remove an element from any specified index of an array using the delete keyword.

```
let numbers = [1, 2, 3, 4, 5];
```

```
// remove one element at index 2
```

```
delete numbers[2]
```

```
console.log(numbers);
```

```
// Output: [ 1, 2, 4, 5 ]
```

JavaScript Multidimensional Array

In JavaScript, multidimensional arrays contain another array inside them.

Example

```
// multidimensional array
```

```
// contains 3 separate arrays as elements
```

```
const data = [[1, 2, 3], [1, 3, 4], [4, 5, 6]];
```

```
console.log(data);
```

```
// Output : [ [ 1, 2, 3 ], [ 1, 3, 4 ], [ 4, 5, 6 ] ]
```

Javascript Array Methods

[JavaScript Array length](#)

Returns the number of elements in an array.

Syntax :

```
arr.length
```

[JavaScript Array reverse\(\)](#)

Returns the array in reverse order.

Syntax :

```
arr.reverse()
```

[JavaScript Array toReversed\(\)](#)

Returns the new array in reverse order.

Syntax :

```
arr.toReversed()
```

[JavaScript Array sort\(\)](#)

Sorts the elements of an array in specific order

Syntax :

```
arr.sort(compareFunction)
```

[JavaScript Array toSorted\(\)](#)

Sorts the elements of an array in specific order and return new array.

Syntax :

```
arr.toSorted(compareFunction)
```

[JavaScript Array fill\(\)](#)

Returns array by filling elements with given value

Syntax

```
arr.fill(value, start, end)
```

[Javascript Array join\(\)](#)

Concatenates the array elements to a string.

Syntax

```
arr.join(separator)
```

[Javascript Array toString\(\)](#)

Returns the string representation of an array

Syntax

```
arr.toString()
```

[JavaScript Array pop\(\)](#)

Removes and returns the last array element

Syntax

```
arr.pop()
```

[JavaScript Array shift\(\)](#)

Removes and returns the first array element

Syntax

```
arr.shift()
```

[JavaScript Array push\(\)](#)

Adds elements to end of array & returns its length

Syntax

```
arr.push(element1, element2, ..., elementN)
```

[JavaScript Array unshift\(\)](#)

Adds elements to start of array and returns length

Syntax

```
arr.unshift(element1, element2, ..., elementN)
```

[JavaScript Array concat\(\)](#)

Returns array by merging given value and/or arrays

Syntax

```
arr.concat(value1, value2, ..., valueN)
```

[JavaScript Array splice\(\)](#)

Returns an array by changing its elements in place

Syntax

```
arr.splice(start, deleteCount, item1, ..., itemN)
```

[JavaScript Array toSpliced\(\)](#)

Returns an array by changing its elements in place and return new array.

Syntax

```
arr.toSpliced(start, deleteCount, item1, ..., itemN)
```

[JavaScript Array lastIndexOf\(\)](#)

Returns the last index of occurrence of an element

Syntax

```
arr.lastIndexOf(searchElement, fromIndex)
```

[JavaScript Array indexOf\(\)](#)

Returns the first index of occurrence of element

Syntax

```
arr.indexOf(searchElement, fromIndex)
```

[JavaScript Array of\(\) Method](#)

Creates a new Array instance from given arguments

Syntax

```
Array.of(element0, element1, ..., elementN)
```

[JavaScript Array slice\(\)](#)

Returns a shallow copy of a portion of an array

Syntax

```
arr.slice(start, end)
```

[JavaScript Array findIndex\(\)](#)

Returns index of element that satisfies condition

Syntax

```
arr.findIndex(callback(element, index))
```

[JavaScript Array find\(\)](#)

Returns first element that satisfies a condition

Syntax

```
arr.find(callback(element, index))
```

[JavaScript Array includes\(\)](#)

Checks if a value exists in an array

Syntax

```
arr.includes(valueToFind, fromIndex)
```

[Javascript Array reduceRight\(\)](#)

Reduces array to single value from right to left

Syntax

```
arr.reduceRight(callback(accumulator, currentValue))
```

[Javascript Array reduce\(\)](#)

Reduces array to single value from left to right

Syntax

```
arr.reduce(callback(accumulator, currentValue))
```

[Javascript Array isArray\(\)](#)

Checks if the given value is a JavaScript Array

Syntax

```
Array.isArray(arr)
```

[Javascript Array filter\(\)](#)

Returns array by filtering elements on given test

Syntax

```
arr.filter(callback(element))
```

[JavaScript Array map\(\)](#)

Returns array by mapping elements using given func

Syntax

```
arr.map(callback(currentValue))
```

[Javascript Array forEach\(\)](#)

Executes the given function on array elements

Syntax

```
arr.forEach(callback(currentValue))
```

[Javascript Array some\(\)](#)

Tests if any element passes given test function

Syntax

```
arr.some(callback(currentValue))
```

[JavaScript Array.every\(\)](#)

Tests if all elements pass the given test function

Syntax

```
arr.every(callback(currentValue))
```

[Javascript Array entries\(\)](#)

Returns iterator containing key/value pair array

Syntax

```
arr.entries()
```

[JavaScript Array keys\(\)](#)

Returns an iterator containing keys of array items

Syntax

```
arr.keys()
```


[JavaScript Array values\(\)](#)

Returns iterator containing values of array items

Syntax

```
arr.values()
```

[Javascript Array.from\(\)](#)

Creates a new Array from array-like object

Syntax

```
Array.from(arraylike)
```

[Javascript Array copyWithin\(\)](#)

Copies and overwrites elements within the array

Syntax

```
arr.copyWithin(target, start, end)
```

[JavaScript Array flat\(\)](#)

Flattens the nested array to given depth

Syntax

```
arr.flat(depth)
```

[JavaScript Array flatMap\(\)](#)

Returns new array by mapping and flattening array

Syntax

```
arr.flatMap(callback(currentValue))
```